

Prime Stack Defect Report

Course: Software Validation, Verification and Security Testing
Application under test: Frost Inventory and Shopping System
Team: Prime Stack
Environment: Local backend `http://localhost:3000` , frontend `http://localhost:4200`
Report date: 14 May 2026

1. Defect Summary

This report records the actual defects confirmed from the project test work, route review, security test design, and ZAP evidence. DEF-001 is marked fixed because the current defect list states it has already been corrected.

ID	Title	Severity	Priority	Status	Related evidence
DEF-001	Missing fields return 500 instead of 400	Medium	High	Fixed	TC-FUNC-002, DEF DOC before/after images
DEF-002	Template literal bug in <code>getPurchaseByUserId</code>	Medium	High	Open	Purchase route/test review
DEF-003	<code>SupplierRoute</code> POST has no error handling	Medium	Medium	Open	Supplier route review
DEF-004	Purchase create fails when <code>userId</code> is passed from test	High	High	Open	TC-FUNC-030
DEF-005	Login route hardcodes status 500	Medium	High	Open	TC-FUNC-010 to TC-FUNC-012
DEF-006	Mass assignment allows any user to register as admin	Critical	Critical	Open	ST-005
DEF-007	<code>/User/users</code> exposes all user data without authentication	High	High	Open	ST-007, route review
DEF-008	XSS payload stored in database without sanitization	Medium	High	Open	ST-003
DEF-009	Missing security headers	Low	Medium	Open	ZAP scan, 14 May 2026

2. Detailed Defects

DEF-001: Missing Fields Return 500 Instead of 400

Field	Value
Severity	Medium
Priority	High
Status	Fixed
Source	Functional test / provided <code>def.docx</code> evidence
Related test	TC-FUNC-002
Endpoint	<code>POST /User/register</code>

Description: When a required registration field such as `email` was missing, the API returned `500 Internal Server Error` . This incorrectly classified a client validation problem as a server failure.

Steps to reproduce:

1. Send `POST /User/register`.
2. Omit the `email` field from the request body.
3. Inspect the response status code.

Expected result: `400 Bad Request` with a clear validation message.

Actual result before fix: `500 Internal Server Error`.

Current result: Fixed. The endpoint now returns the expected validation response.

Evidence: Images extracted from `/home/ahmet/Desktop/def.docx` are stored under `docs/testing/evidence/def-doc/word/media/`.

```
✖ Registration error: ConflictError: Username already exists
  at UserService.ensureUniqueUsername (/home/ahmet/WebstormProjects/meanProject/backend/domain/user/UserService.js:131:19)
  at processTicksAndRejections (node:internal/process/task_queues:95:5)
  at UserService.createUser (/home/ahmet/WebstormProjects/meanProject/backend/domain/user/UserService.js:24:13)
  at UserApplication.createUser (/home/ahmet/WebstormProjects/meanProject/backend/application/UserApplication.js:17:20)
  at /home/ahmet/WebstormProjects/meanProject/backend/api/UserRoute.js:52:24 {
    statusCode: 409
  }

59 |
60 |   } catch (error) {
> 61 |     console.error('✖ Registration error:', error);
    |           ^
62 |     const statusCode = error.statusCode || 500;
63 |
64 |     res.status(statusCode).json({

    at error (api/UserRoute.js:61:17)

FAIL tests/user.test.js
User Registration
  ✓ should register a new user successfully (98 ms)
  ✖ should reject registration with missing email (24 ms)
  ✓ should reject registration with duplicate email (7 ms)

  ● User Registration › should reject registration with missing email

    expect(received).toBe(expected) // Object.is equality

    Expected: 400
    Received: 500

    27 |         });
    28 |
> 29 |         expect(response.status).toBe(400);
      |                               ^
    30 |       });
    31 |
    32 |     test('should reject registration with duplicate email', async () => {

    at Object.toBe (tests/user.test.js:29:33)

Test Suites: 1 failed, 1 total
Tests:       1 failed, 2 passed, 3 total
Snapshots:   0 total
Time:        0.818 s
Ran all test suites.
Force exiting Jest: Have you considered using '--detectOpenHandles' to detect async operations that kept running after all tests finished?
```

DEF-001 evidence overview

```

5
6
7 export default class Email{ Show usages  ahmet-kilicarslan *
8 constructor(value) { Show usages  ahmet-kilicarslan *
9
10     if (!value) {
11         throw new Error( message: "Email required");
12     }
13
14     const emailRegex:RegExp =
15
16     if(!emailRegex.test(value),
17         throw new Error( message: "Invalid email format");
18     }
19
20     this._value = value.toLowerCase().trim();
21
22     Object.freeze(this);//makes it immutable
23
24 }
25
26 get value() { Show usages  ahmet-kilicarslan
27     return this._value;
28 }
29
30 equals(otherValue) { no usages  ahmet-kilicarslan
31     return otherValue instanceof Email && this._value === otherValue;
32 }
33
34 }

```

DEF-001 before fix

```

1 import {ValidationError} from "../../Utilities/errors.js";
2
3
4 export default class Email{ Show usages  ahmet-kilicarslan *
5 constructor(value) { Show usages  ahmet-kilicarslan *
6
7     if (!value) {
8         throw new ValidationError("Email required");
9     }
10
11     const emailRegex:RegExp = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
12
13     if(!emailRegex.test(value)) {
14         throw new ValidationError("Invalid email format");
15     }
16
17     this._value = value.toLowerCase().trim();
18
19     Object.freeze(this);//makes it immutable
20
21 }
22
23 get value() { Show usages  ahmet-kilicarslan
24     return this._value;
25 }
26
27 equals(otherValue) { no usages  ahmet-kilicarslan
28     return otherValue instanceof Email && this._value === otherValue;
29 }
30
31 }
32

```

DEF-001 after fix

DEF-002: Template Literal Bug in `getPurchaseByUserId`

Field	Value
Severity	Medium
Priority	High
Status	Open
Source	Purchase service/repository review
Related area	Purchase history

Description: The purchase retrieval logic contains a template literal issue in `getPurchaseByUserId`. When the query is constructed incorrectly, the method can fail or produce an invalid SQL/order expression.

Steps to reproduce:

1. Start the backend.
2. Request purchase history using `GET /Purchase/byUserId/:userId`.
3. Use both `order=asc` and `order=desc`.

Expected result: The endpoint returns the selected user's purchases in the requested order.

Actual result: The method can fail because the query string is not built correctly.

Suggested fix: Correct the template literal syntax in the purchase repository/service and keep the order value allow-listed to `asc` or `desc`.

DEF-003: `SupplierRoute` POST Has No Error Handling

Field	Value
Severity	Medium
Priority	Medium
Status	Open
Source	Route review
Endpoint	<code>POST /Supplier/</code>
Related tests	TC-FUNC-023, TC-FUNC-024

Description: The supplier creation route directly awaits `supplierApplication.createSupplier(req.body)` and returns the result without a `try/catch`. If supplier validation or duplicate checks throw an error, Express may return an uncontrolled server error instead of a clear API response.

Steps to reproduce:

1. Send `POST /Supplier/` with invalid or duplicate supplier data.
2. Observe whether the API returns a controlled error response.

Expected result: Invalid supplier input should return a controlled `400` or `409` response.

Actual result: The route has no local error handling and can return an unstructured error.

Suggested fix: Wrap supplier creation in `try/catch`, map duplicate supplier errors to `409`, validation errors to `400`, and unexpected errors to `500`.

DEF-004: Purchase Create Fails When `userId` Is Passed From Test

Field	Value
Severity	High
Priority	High
Status	Open
Source	Functional test
Related test	TC-FUNC-030
Endpoint	POST /Purchase/create

Description: The purchase creation flow fails when `userId` is supplied from the test setup. This blocks automated verification of the normal purchase workflow.

Steps to reproduce:

1. Select an existing user id and product id from the database.
2. Send `POST /Purchase/create` with `userId`, `totalAmount`, and a `products` array.
3. Observe the response.

Expected result: Status `201`, with a purchase object and purchased-product records returned.

Actual result: Purchase creation fails for the test-provided `userId`.

Suggested fix: Validate the exact request shape expected by `PurchaseApplication.createPurchase`, confirm user id foreign-key compatibility, and add clear route-level validation before inserting purchase records.

DEF-005: Login Route Hardcodes Status 500

Field	Value
Severity	Medium
Priority	High
Status	Open
Source	Functional tests / route review
Related tests	TC-FUNC-010, TC-FUNC-011, TC-FUNC-012
Endpoint	POST /User/login

Description: The login route returns `500` for failed login cases. Wrong passwords, unknown users, and missing credentials are expected client/authentication failures, not internal server errors.

Steps to reproduce:

1. Send `POST /User/login` with a wrong password.
2. Send `POST /User/login` with an unknown username.
3. Send `POST /User/login` with an empty body.

Expected result: `401 Unauthorized` for invalid credentials and `400 Bad Request` for missing credentials.

Actual result: The route hardcodes or maps these cases to `500`.

Suggested fix: Return status codes based on error type: `400` for missing input, `401` for invalid credentials, and `500` only for unexpected server failures.

DEF-006: Mass Assignment Allows Any User to Register as Admin

Field	Value
Severity	Critical
Priority	Critical
Status	Open
Source	Security test design
Related test	ST-005
Endpoint	POST /User/register

Description: A user can include privileged fields such as `role: "admin"` in the registration payload. If the backend accepts that field, any unauthenticated user can create an administrator account.

Steps to reproduce:

1. Send `POST /User/register`.
2. Include a normal username, email, password, and `"role": "admin"` in the body.
3. Inspect the created user's role.

Expected result: The backend ignores client-provided role values and assigns a safe default role.

Actual result: The mass assignment test indicates that admin role assignment is possible.

Suggested fix: Use an explicit allow-list of registration fields. Do not accept `role`, `id`, or other privileged fields from public registration requests.

DEF-007: /User/users Exposes All User Data Without Authentication

Field	Value
Severity	High
Priority	High
Status	Open
Source	Security test design / route review
Related test	ST-007
Endpoint	GET /User/users

Description: The user list route is publicly reachable without authentication or admin authorization. This can expose user records and supports account enumeration.

Steps to reproduce:

1. Start the backend.
2. Do not log in.
3. Send `GET /User/users`.

Expected result: `401 Unauthorized` for unauthenticated requests and `403 Forbidden` for authenticated non-admin users.

Actual result: Route review shows the endpoint calls `userApplication.getAllUsers()` without `requireAuth` or an admin guard.

Suggested fix: Add authentication middleware and an admin-role check to `/User/users`. Return only safe user fields.

DEF-008: XSS Payload Stored in Database Without Sanitization

Field	Value
Severity	Medium
Priority	High
Status	Open
Source	Security test design
Related test	ST-003
Endpoint	POST /User/register

Description: The application accepts an XSS payload in user-controlled input and stores it in the database. If that value is rendered later without escaping, it may execute script in a user's browser.

Steps to reproduce:

1. Send POST /User/register .
2. Use <script>alert("xss")</script> as the username.
3. Check whether the value is rejected, sanitized, or stored unchanged.

Expected result: The payload is rejected or safely encoded before storage/display.

Actual result: The XSS payload is stored without sanitization.

Suggested fix: Validate username characters, reject HTML/script tags, and rely on frontend output encoding when displaying user-controlled values.

DEF-009: Missing Security Headers

Field	Value
Severity	Low
Priority	Medium
Status	Open
Source	ZAP scan
Tool	ZAP by Checkmarx 2.17.0
Scan date	14 May 2026
Related alerts	CSP no-fallback directives, missing X-Content-Type-Options , X-Powered-By: Express

Description: ZAP identified missing or weak security headers on backend responses. These are hardening issues rather than direct exploit proof, but they reduce browser-side protections and reveal server implementation details.

Evidence from ZAP:

Header issue	Risk	Evidence
CSP missing no-fallback directives	Medium	Content-Security-Policy: default-src 'none' without frame-ancestors and form-action
X-Content-Type-Options missing	Low	/Product/ response does not include nosniff

Header issue	Risk	Evidence
X-Powered-By exposed	Low	Response contains X-Powered-By: Express

Steps to reproduce:

1. Run ZAP automated scan against `http://localhost:3000/Product/` .
2. Review passive alerts.
3. Inspect headers on `/Product/` and missing routes such as `/robots.txt` .

Expected result: Security headers are present and technology disclosure is suppressed.

Actual result: ZAP reports missing security headers and Express disclosure.

Suggested fix: Add centralized Express security middleware. At minimum, disable `x-powered-by` , add `X-Content-Type-Options: nosniff` , and configure CSP explicitly.

3. Recommended Fix Order

1. Fix DEF-006 immediately because public admin registration is a critical privilege escalation.
2. Fix DEF-007 by protecting `/User/users` with authentication and admin authorization.
3. Fix DEF-004 so the main purchase workflow can be tested reliably.
4. Fix DEF-005 and DEF-003 to improve API error handling and test clarity.
5. Fix DEF-002 to stabilize purchase history.
6. Fix DEF-008 by validating/sanitizing user input.
7. Fix DEF-009 with centralized security headers.
8. Keep DEF-001 marked fixed and retain its before/after evidence for the final report.